Custom Linux Shell and Multithreaded Linux Applicati Using POSIX Threads and Mutexes

Team ID: 8 | Section: 10 | Members: 2520090170 – S Svara Haasini, 2520090227 – B. Sudharshini, 2520090190 – Sumanjali | Guide: Mr. M. Raghupathi

Abstract

- This project integrates a custom Linux shell with a multithreaded C application.
- The shell executes Linux commands and launches the project using OS services such as process creation and program execution.
- The main application demonstrates race conditions on a shared counter and shows how POSIX mutex synchronization ensures correct, thread-safe output.
- Key outcome: clear practical demonstration of concurrency, shared data hazards, and mutual exclusion in Linux.

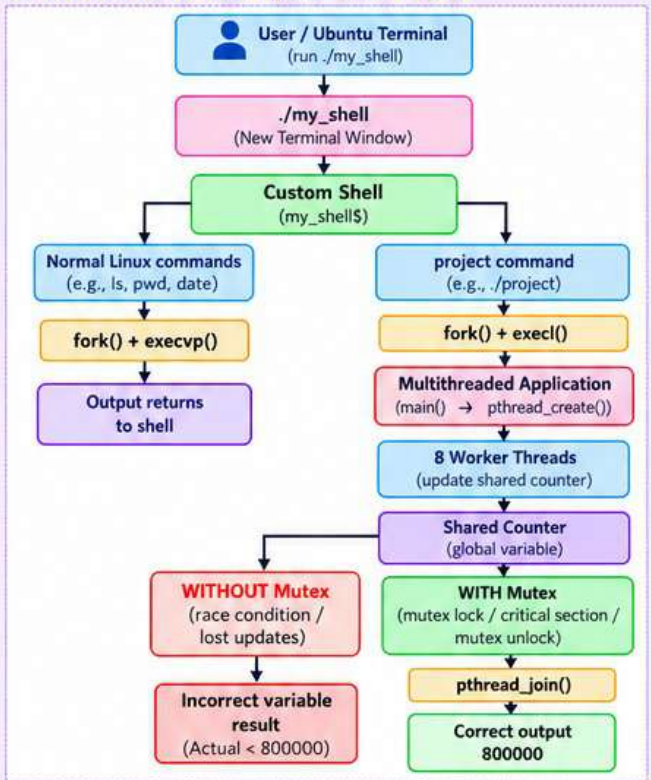
Tools & Technologies

- **Programming:** C
- **Platform:** Linux / Ubuntu (WSL)
- **OS Concepts:** user-space shell, processes, threads, shared data, critical section, race condition, mutual exclusion
- **APIs / Functions:** fork(), execvp(), execl(), waitpid(), chdir(), pthread_create(), pthread_join(), pthread_mutex_init(), pthread_mutex_lock(), pthread_mutex_unlock(), pthread_mutex_destroy(), sched_yield()
- **Tools:** GCC, Make, Git & GitHub, VS Code / Ubuntu terminal

Problem Statement & Objectives

- **Problem:** Concurrent threads updating shared data without synchronization can cause race conditions, lost updates, and incorrect results.
- **Objectives:**
  1. Build a custom Linux shell to execute commands in user space.
  2. Create a multithreaded program using POSIX threads.
  3. Demonstrate the difference between unsafe shared-data access and mutex-protected access.
  4. Integrate shell execution and synchronization concepts into one OSSP project.
- **Scope:** Covers shell commands, process creation, thread creation, shared counter, mutex locking, and result comparison.

System Workflow / Architecture



Existing vs Proposed System

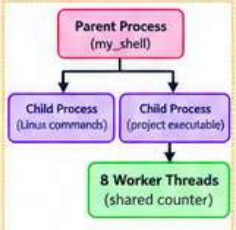
Aspect	Basic / Existing Approach	Propesed Project
Process handling	Normal command usage only	Custom shell manages command execution with fork/exec/wait
Synchronization	Shared data may be updated unsafely	Mutex protects the critical section
Result behavior	Non-deterministic / lost updates possible	Correct and thread-safe output
Learning value	Concepts seen separately	Processes, threads, and synchronization integrated in one project
Key gap	Hard to visualize race conditions	Direct comparison of without-mutex vs with-mutex execution

Feasibility Study

- Technical: feasible with C, GCC, POSIX threads, and Ubuntu/WSL.
- Economic: low cost; uses free open-source tools.
- Operational: easy to run using make and ./my_shell; suitable for lab demonstration.
- Schedule: designed and tested within the semester project timeline.

Process, IPC & Resource Design

- **Parent process:** custom shell (./my_shell)
- **Child processes:** external commands and the project executable
- **Worker threads:** 8 POSIX threads inside the project process
- **Shared resource:** global counter variable
- **Critical section:** counter update
- **Resource protection:** pthread mutex
- **Note:** no explicit pipes/FIFOs/signals are implemented in this version; focus is on process management and thread synchronization.



Implementation Screenshots

Screenshot 1: Custom Shell Terminal

```
./my_shell
========================================
        Custom Linux Shell (my_shell)
        Type 'help' for available commands
========================================
my_shell$ help
Built-in commands: help, exit, cd
my_shell$ pwd
/home/student/ossp
my_shell$ ls
my_shell  project   Makefile   README.md
my_shell$ date
Thu Apr 24 15:32:10 IST 2025
my_shell$ whoami
student
my_shell$ project
```

Screenshot 2: Project Output (Multithreaded Application)

```
./project
===== Multithreaded Counter with POSIX Threads =====
1. Run WITHOUT Mutex
2. Run WITH Mutex
3. Run BOTH
Enter your choice (1/2/3): 3

--- Running WITHOUT Mutex ---
Expected value : 800000
Actual value   : 723514
Status         : RACE CONDITION DETECTED

--- Running WITH Mutex ---
Expected value : 800000
Actual value   : 800000
Status         : CORRECT / THREAD SAFE
```

Results & Testing

- **Input choice 1:** demonstrates race condition; actual result varies and is usually less than 800000.
- **Input choice 2:** mutex-protected execution gives the exact expected result 800000.
- **Input choice 3:** compares both modes in one run.
- **Additional tests:** shell commands, cd behavior, invalid choice handling.
- **Observation:** synchronization ensures correctness for shared data access.

Conclusion

- The project successfully combines shell programming, process control, and multithreading concepts.
- It clearly demonstrates how race conditions occur and how POSIX mutexes ensure thread-safe outputs.
- It strongly maps to OSSP topics such as shell, process management, threads, and synchronization.

Future Scope

- Extend the shell with pipes, I/O redirection, and history.
- Add signals and job control features.
- Compare mutex with semaphores and condition variables.
- Study performance with different thread counts or thread-pool designs.

QR: GitHub Repo

References: [1] Linux man-pages [2] Programming with POSIX Threads / course material [3] GCC / GNU documentation